

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium) Assessment

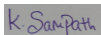
Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I K. Sampath Chaitanya certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Introduction

This report presents a scenario-based analysis of ten real-world database situations, each mapped to a core concept in relational database design: normalization (1NF–5NF), functional dependencies and candidate keys, BCNF decomposition, SQL query design using joins and subqueries, views and access control, integrity constraints, and query optimization. Each scenario follows a consistent structure — problem understanding, identification of key issues, application of relevant DBMS concepts, the solution/decision, and justification — in line with the assessment rubric for CO2.

Question 1: Normalizing a University Student–Course Table (Up to 3NF)

Scenario

A university stores all student and course data in a single flat table containing student ID, student name, course ID, course name, instructor, instructor office, and department. Because one student can enroll in multiple courses, and one course can be taught by only one instructor, this design mixes several independent facts into one relation.

Analysis of the Unnormalized Table

Sample structure:

StudentID	StudentName	CourseID	CourseName	Instructor	InstructorOffice	Dept
S1	Arjun	C101	DBMS	Dr. Rao	B-204	CSE
S1	Arjun	C102	OS	Dr. Iyer	B-210	CSE
S2	Divya	C101	DBMS	Dr. Rao	B-204	CSE

Normalization Issues Identified

- Repeating groups: a student appears once per enrolled course, duplicating StudentName.
- Transitive dependency: InstructorOffice depends on Instructor, which depends on CourseID, not directly on the key — violates 3NF.
- Update anomaly: changing an instructor's office requires updating every row that references that instructor.
- Insertion anomaly: a new course cannot be stored until at least one student enrolls in it.
- Deletion anomaly: removing the only student enrolled in a course erases all information about that course.

Redesign to 3NF

Functional dependencies identified:

- StudentID $\rightarrow$ StudentName
- CourseID $\rightarrow$ CourseName, Instructor, Dept
- Instructor $\rightarrow$ InstructorOffice
- (StudentID, CourseID) $\rightarrow$ (all attributes, via the above)

Decomposed Relations (3NF)

Relation	Attributes	Key
STUDENT	StudentID, StudentName	StudentID (PK)
COURSE	CourseID, CourseName, Instructor, Dept	CourseID (PK), Instructor (FK)
INSTRUCTOR	InstructorID/Name, InstructorOffice	Instructor (PK)
ENROLLMENT	StudentID, CourseID	(StudentID, CourseID) composite PK; FKs to STUDENT, COURSE

This decomposition removes the repeating group (1NF), eliminates the partial dependency on the composite key (2NF), and removes the transitive dependency of InstructorOffice on CourseID via Instructor (3NF). Each non-key attribute now depends only on the whole key of its own relation, and lossless join is preserved because ENROLLMENT retains the foreign keys needed to reconstruct the original data via joins.

Question 2: Employees Working in Multiple Departments — SQL Query Design

Scenario

A company database contains EMPLOYEE and DEPARTMENT information, and (in organizations that allow it) an employee may be associated with more than one department through project assignments. Management wants to retrieve the list of employees who work in more than one department.

Schema Assumption

Table	Key Columns
EMPLOYEE	EmpID (PK), EmpName
DEPARTMENT	DeptID (PK), DeptName
EMP_DEPT (bridge table for many-to-many)	EmpID (FK), DeptID (FK)

Query Design

Because an employee-to-department relationship here is many-to-many, a bridge table is required. Employees in more than one department are found by grouping on EmpID and filtering groups with more than one distinct department:

```
SELECT e.EmpID, e.EmpName, COUNT(DISTINCT ed.DeptID) AS DeptCount
FROM EMPLOYEE e
JOIN EMP_DEPT ed ON e.EmpID = ed.EmpID
GROUP BY e.EmpID, e.EmpName
HAVING COUNT(DISTINCT ed.DeptID) > 1;
```

An equivalent version that also lists the department names uses a self-join style correlated subquery:

```
SELECT DISTINCT e.EmpID, e.EmpName, d.DeptName
FROM EMPLOYEE e
JOIN EMP_DEPT ed ON e.EmpID = ed.EmpID
JOIN DEPARTMENT d ON ed.DeptID = d.DeptID
WHERE e.EmpID IN (
    SELECT EmpID FROM EMP_DEPT
    GROUP BY EmpID HAVING COUNT(DISTINCT DeptID) > 1
);
```

Justification

- GROUP BY with HAVING is preferred over a correlated subquery for the core aggregation because it evaluates the count once per employee and scales better on large tables.
- The bridge (junction) table EMP_DEPT correctly models the many-to-many relationship; without it the design would force repeating groups inside EMPLOYEE, violating 1NF.
- COUNT(DISTINCT ...) avoids over-counting employees who have duplicate rows for the same department (e.g., multiple projects in one department).
- The subquery variant is useful when descriptive columns (like DeptName) must be displayed alongside the filter, which HAVING alone cannot do.

Question 3: Candidate Keys and BCNF Check with Decomposition

Scenario

Consider a relation $R(A, B, C, D)$ representing course-room-time bookings, with the attributes: $A = \text{Student}$, $B = \text{Course}$, $C = \text{Instructor}$, $D = \text{Room}$. The following functional dependencies are given:

- FD1: $A, B \rightarrow C$ (a student's grade/registration for a course determines the instructor)
- FD2: $C \rightarrow D$ (each instructor always teaches in one fixed room)
- FD3: $A, B \rightarrow D$ (implied by FD1 and FD2 via transitivity)

Step 1: Candidate Key Determination

Using attribute closure, $(A,B)^+ = \{A, B, C, D\}$ — because $A,B \rightarrow C$, and then $C \rightarrow D$ can be added. Since no attribute is repeated on the left that yields a smaller closure covering all attributes, (A, B) is the only candidate key of R .

Step 2: BCNF Check

A relation is in BCNF only if, for every non-trivial FD $X \rightarrow Y$, X is a superkey. Checking each FD:

FD	Determinant a superkey?	BCNF Violation?
$A, B \rightarrow C$	Yes — (A,B) is the candidate key	No
$C \rightarrow D$	No — C alone is not a superkey	Yes, violates BCNF

Because $C \rightarrow D$ has a determinant (C) that is not a superkey, R is not in BCNF (though it satisfies 3NF, since D is a prime-attribute-independent non-key attribute transitively dependent through C , and 3NF permits this exception when C is itself a determinant of a key — this is the classic "3NF-but-not-BCNF" case).

Step 3: BCNF Decomposition

Decompose using the violating FD $C \rightarrow D$:

- $R_1(C, D)$ — with C as the key, satisfies $C \rightarrow D$ trivially as a superkey dependency.
- $R_2(A, B, C)$ — with (A, B) as the key, satisfies $A, B \rightarrow C$ as a superkey dependency.

Both R_1 and R_2 are now in BCNF, and the decomposition is lossless ($R_1 \cap R_2 = C$, which is a key of R_1 , satisfying the lossless-join test) and dependency-preserving (both original FDs are enforceable within the two relations).

Question 4: Views and Constraints for Secure Banking Data Access

Scenario

A banking system must expose different levels of information to different user roles — tellers, loan officers, branch managers, and customers — while protecting sensitive fields such as account balance history, SSN/Aadhaar numbers, and full transaction logs.

Design Approach

1. Role-Based Views

```
-- View for customer self-service (limited columns, own rows only)
CREATE VIEW CustomerAccountView AS
SELECT AccountID, AccountType, Balance, LastTransactionDate
FROM ACCOUNTS
WHERE CustomerID = CURRENT_USER_ID();

-- View for tellers (no SSN, no full transaction history)
CREATE VIEW TellerAccountView AS
SELECT AccountID, CustomerName, AccountType, Balance
FROM ACCOUNTS a JOIN CUSTOMERS c ON a.CustomerID = c.CustomerID;

-- View for branch managers (aggregate/reporting access)
CREATE VIEW BranchSummaryView AS
SELECT BranchID, COUNT(*) AS TotalAccounts, SUM(Balance) AS TotalDeposits
FROM ACCOUNTS GROUP BY BranchID;
```

2. Integrity and Security Constraints

Constraint Type	Purpose	Example
PRIMARY KEY	Uniquely identifies each account/customer	PRIMARY KEY (AccountID)
FOREIGN KEY	Maintains referential integrity between customers and accounts	FOREIGN KEY (CustomerID) REFERENCES CUSTOMERS
CHECK	Enforces business rules (e.g., balance cannot go negative)	CHECK (Balance >= 0)
NOT NULL	Ensures mandatory fields like AccountType are always present	AccountType VARCHAR(20) NOT NULL
GRANT/REVOKE	Restricts which roles can SELECT/UPDATE which views	GRANT SELECT ON TellerAccountView TO teller_role
WITH CHECK OPTION	Prevents a view from being used to insert/update rows outside its own filter	CREATE VIEW ... WITH CHECK OPTION

Justification

- Views enforce column-level and row-level security without duplicating data — each role queries only what it is permitted to see.
- WITH CHECK OPTION prevents a teller from using an updatable view to modify rows that fall outside the view's WHERE clause, closing a common privilege-escalation gap.
- CHECK and NOT NULL constraints protect data integrity at the storage layer, independent of which view or application accesses it.
- GRANT/REVOKE on the view (not the base table) means the base ACCOUNTS table itself remains inaccessible to lower-privilege roles.

Question 5: Eliminating Redundancy in a Sales Database (3NF/BCNF)

Scenario

A sales database stores order data in a single SALES table with columns: OrderID, ProductID, ProductName, ProductPrice, CustomerID, CustomerName, CustomerCity, Quantity, and SalesRep. This flat structure repeats product and customer details on every order line.

Anomaly Analysis

- Update anomaly: changing ProductPrice requires updating every order row that references that product.
- Insertion anomaly: a new product cannot be added to the catalog until it is actually ordered.
- Deletion anomaly: deleting the last order for a customer removes all record of that customer's CustomerCity.

Functional Dependencies

- OrderID → CustomerID, SalesRep (order-level facts)
- ProductID → ProductName, ProductPrice
- CustomerID → CustomerName, CustomerCity
- (OrderID, ProductID) → Quantity

Normalized Design (3NF)

Relation	Attributes	Key
CUSTOMER	CustomerID, CustomerName, CustomerCity	CustomerID (PK)
PRODUCT	ProductID, ProductName, ProductPrice	ProductID (PK)
ORDERS	OrderID, CustomerID, SalesRep, OrderDate	OrderID (PK), CustomerID (FK)
ORDER_ITEMS	OrderID, ProductID, Quantity	(OrderID, ProductID) composite PK

Each relation now has every non-key attribute dependent on the whole key and nothing but the key. A BCNF check confirms every determinant (CustomerID, ProductID, OrderID, and the composite key) is a candidate key of its own relation, so the design is also in BCNF — no further decomposition is required, and redundant repetition of product/customer data across order lines is eliminated.

Question 6: Customer–Order Queries Using Joins and Subqueries

Scenario

Given CUSTOMERS(CustomerID, Name, City) and ORDERS(OrderID, CustomerID, OrderDate, Amount), management wants to identify frequent customers and related order insights.

Query A — Customers with more than 3 orders (frequent customers)

```
SELECT      c.CustomerID,      c.Name,      COUNT(o.OrderID)      AS      OrderCount
FROM
      CUSTOMERS      c
JOIN      ORDERS      o      ON      c.CustomerID      =      o.CustomerID
GROUP
      BY      c.CustomerID,      c.Name
HAVING
      COUNT(o.OrderID)      >      3
ORDER BY OrderCount DESC;
```

Query B — Customers whose total spend exceeds the average customer spend (subquery)

```
SELECT      c.CustomerID,      c.Name,      SUM(o.Amount)      AS      TotalSpend
FROM      CUSTOMERS      c      JOIN      ORDERS      o      ON      c.CustomerID      =      o.CustomerID
GROUP
      BY      c.CustomerID,      c.Name
HAVING
      SUM(o.Amount)      >      (
      SELECT
            AVG(CustomerTotal)
      FROM      (
            SELECT
                  SUM(Amount)      AS      CustomerTotal
            FROM      ORDERS      GROUP      BY      CustomerID
            )
      )
);
```

Query C — Customers who placed at least one order in every product category (division-style subquery)

```
SELECT DISTINCT o.CustomerID
FROM ORDERS o
WHERE NOT EXISTS (
    SELECT Category
    EXCEPT
    SELECT p.Category
    FROM ORDER_ITEMS oi
    JOIN PRODUCT p ON oi.ProductID = p.ProductID
    WHERE oi.OrderID IN (SELECT OrderID FROM ORDERS WHERE CustomerID =
o.CustomerID)
);
```

Justification

- INNER JOIN with GROUP BY/HAVING is the natural fit for aggregate frequency questions (Query A).
- A nested subquery is needed in Query B because the comparison value (average spend) is itself derived from an aggregate over all customers, which cannot be expressed in a single-level GROUP BY.
- The NOT EXISTS / EXCEPT pattern in Query C implements relational division, correctly identifying customers who satisfy an 'all categories' condition — a plain JOIN cannot express this universal quantification.

Question 7: Multi-Valued Dependencies and Decomposition to 4NF

Scenario

A relation EMP_SKILLS_LANGUAGES(Employee, Skill, Language) records that an employee has a set of skills and, independently, a set of languages spoken. These two facts are unrelated to each other but both depend only on Employee, causing every skill to be paired with every language for that employee.

Analysis

Sample data illustrating the redundancy:

Employee	Skill	Language
Ravi	SQL	English
Ravi	SQL	Tamil
Ravi	Java	English
Ravi	Java	Tamil

This is a classic multi-valued dependency (MVD) situation: Employee $\twoheadrightarrow$ Skill and Employee $\twoheadrightarrow$ Language, where Skill and Language are independent of each other. The relation is already in BCNF (the only key is the full attribute set, so there is no partial/transitive FD violation), but it still suffers redundancy because unrelated multi-valued facts are combined in one table — every skill must be repeated for every language.

Decomposition to 4NF

4NF requires that for every non-trivial MVD $X \twoheadrightarrow Y$, X must be a superkey, and critically, that independent multivalued facts are not combined. Decompose into two relations, each capturing one independent multi-valued fact:

Relation	Attributes	Key
EMP_SKILL	Employee, Skill	(Employee, Skill)
EMP_LANGUAGE	Employee, Language	(Employee, Language)

This decomposition is lossless — the original data is exactly reconstructed by taking the Cartesian product (natural join) of EMP_SKILL and EMP_LANGUAGE per employee — and each relation is now in 4NF because it contains only a single independent multi-valued fact per employee, eliminating the redundant pairing.

Question 8: Evaluating a Schema for Missing Integrity Constraints

Scenario

A database designer created tables for an inventory system without adequate constraints. As a result, orders reference non-existent products, negative stock quantities appear, duplicate SKU codes exist, and some rows have NULL prices.

Evaluation of the Flawed Schema

Observed Problem	Root Cause	Missing Constraint
Orders reference deleted/non-existent products	No link enforcement between ORDERS and PRODUCTS	FOREIGN KEY with referential action
Negative stock quantities	No range validation on Quantity	CHECK (Quantity >= 0)
Duplicate SKU codes	SKU not enforced as unique	UNIQUE constraint on SKU
NULL prices	Price column allows nulls	NOT NULL, and CHECK (Price > 0)
Orphaned rows after deletes	No cascade/restrict rule defined	ON DELETE CASCADE / RESTRICT

Recommended Schema Corrections

ALTER	ADD	CONSTRAINT	uq_sku	UNIQUE	(SKU),
ALTER	COLUMN	Price	SET	NOT	NULL,
ADD	CONSTRAINT	chk_price	CHECK	(Price	> 0),
ADD	CONSTRAINT	chk_qty	CHECK	(Quantity	>= 0);
ALTER					
ALTER					
ADD	CONSTRAINT	fk_product	FOREIGN	KEY	(ProductID)
	REFERENCES				PRODUCTS(ProductID)
	ON DELETE	RESTRICT	ON UPDATE	CASCADE;	

Justification

- FOREIGN KEY with ON DELETE RESTRICT prevents a product from being deleted while active orders still reference it, avoiding orphaned order records.
- CHECK constraints enforce business rules (non-negative stock, positive price) directly at the database layer so no application bug can bypass them.
- UNIQUE on SKU guarantees entity integrity for a natural business key, independent of the surrogate ProductID.
- NOT NULL on Price closes the gap that allowed silent missing-price rows to enter the catalog.

Question 9: Join Dependencies and Decomposition to 5NF

Scenario

A relation SUPPLY(Supplier, Part, Project) records that a supplier can supply a part to a project, but only when the supplier supplies that part, the part is used in that project, and the supplier supplies that project — i.e., the three-way relationship cannot be reconstructed from any two independent pairwise relationships alone. This is the classic setting for a join dependency (JD).

Analysis

Suppose SUPPLY is already in BCNF and 4NF (no simple MVDs exist), yet redundancy remains because the ternary relationship is really governed by a join dependency: $JD(R_1, R_2, R_3)$ where $R_1(\text{Supplier, Part})$, $R_2(\text{Part, Project})$, $R_3(\text{Supplier, Project})$. If decomposing into these three pairwise relations and rejoining them reproduces exactly the original SUPPLY data (no spurious rows), then the join dependency holds and the ternary table can be safely decomposed.

Test for 5NF

The relation is in 5NF if it has no join dependency other than the trivial one implied by a candidate key. To test, we decompose into the three binary projections and recombine with natural joins; if the recombination equals the original relation exactly, the JD is confirmed and the decomposition is valid and lossless.

Decomposition to 5NF

Relation	Attributes	Key
SUPPLIER_PART	Supplier, Part	(Supplier, Part)
PART_PROJECT	Part, Project	(Part, Project)
SUPPLIER_PROJECT	Supplier, Project	(Supplier, Project)

This decomposition removes redundancy that would otherwise force re-storing every valid (Supplier, Part, Project) combination as a single fact, when in reality it is derivable from three independent, smaller relationships. The original ternary data is reconstructed with:

```
SELECT      sp.Supplier,      sp.Part,      pp.Project
FROM      SUPPLIER_PART      sp
JOIN      PART_PROJECT      pp      ON      sp.Part      =      pp.Part
JOIN      SUPPLIER_PROJECT      spr      ON      sp.Supplier      =      spr.Supplier
      AND pp.Project = spr.Project;
```

Because the three-way natural join of the projections reproduces the original relation with no spurious tuples, the decomposition satisfies the lossless-join property required for 5NF.

Question 10: Analyzing and Optimizing Inefficient SQL Queries

Scenario

A reporting system runs slow, inefficient queries against large ORDERS and CUSTOMERS tables — for example, a query that filters using a function on an indexed column, uses SELECT *, and relies on a correlated subquery inside a WHERE clause evaluated once per outer row.

Original (Inefficient) Query

```
SELECT *
FROM ORDERS o
WHERE YEAR(o.OrderDate) = 2025
AND o.CustomerID IN (
    SELECT CustomerID FROM CUSTOMERS
    WHERE City = 'Chennai'
);
```

Issues Identified

- YEAR(OrderDate) = 2025 wraps the column in a function, preventing the query optimizer from using any index on OrderDate (a 'non-sargable' predicate).
- SELECT * retrieves unused columns, increasing I/O and network transfer.
- The IN (subquery) form can, on some optimizers, be evaluated as a correlated/nested loop rather than a set-based join, especially without a supporting index on CUSTOMERS.City.
- No index exists on ORDERS.CustomerID or CUSTOMERS.City, forcing full table scans.

Optimized Query

```
SELECT      o.OrderID,      o.OrderDate,      o.Amount,      o.CustomerID
FROM      CUSTOMERS      c      JOIN      ORDERS      o
ON      o.CustomerID      =      c.CustomerID
WHERE      o.OrderDate      >=      '2025-01-01'
      AND      o.OrderDate      <      '2026-01-01'
      AND      c.City      =      'Chennai';

--      Supporting      indexes:
CREATE      INDEX      idx_orders_date      ON      ORDERS(OrderDate);
CREATE      INDEX      idx_orders_custid      ON      ORDERS(CustomerID);
CREATE INDEX idx_customers_city ON CUSTOMERS(City);
```

Justification for Each Optimization

- Rewriting YEAR(OrderDate)=2025 as a range condition (>= and <) is sargable, allowing the optimizer to use a B-tree index range scan on OrderDate instead of a full scan.
- Replacing SELECT * with only the required columns reduces I/O and can enable index-only (covering) scans.
- Converting the subquery to an explicit JOIN lets the optimizer choose the most efficient join algorithm (hash join, merge join, or index nested-loop) based on table statistics.
- Indexes on the join column (CustomerID) and the filter column (City) let the optimizer avoid full scans of both large tables.
- Overall, these changes reduce query cost from a full Cartesian-style scan-and-filter pattern to an index-driven, set-based execution plan, which scales far better as table size grows.

Conclusion

Across all ten scenarios, three recurring DBMS principles emerge: (1) normalization — from 1NF through 5NF — systematically removes redundancy and the anomalies it causes by ensuring every non-key attribute depends on the whole key, nothing but the key, and no independent multi-valued or join-dependent facts are merged into one relation; (2) SQL query design — using joins, subqueries, GROUP BY/HAVING, and set operators — provides the tools to answer complex business questions correctly and efficiently when the underlying schema is well normalized; and (3) constraints and views together enforce both data integrity and controlled, role-appropriate access without duplicating data. Applying these principles converts ad hoc, anomaly-prone table designs into robust, secure, and performant relational databases.

References: Elmasri & Navathe, Fundamentals of Database Systems; Silberschatz, Korth & Sudarshan, Database System Concepts; standard SQL (ISO/IEC 9075) documentation.